

生成 AI 利用申告書

- 氏名：塩澤 拓海
- 学籍番号：[TODO: 学籍番号を記入]
- プログラムファイル名：HCK02_01.ino
- 使用した AI ツール：Claude Code (Opus 4.7)

1. 何に使ったか

例題 2 (work3.ino) をベースに、Arduino のシリアルプロッタで sin 波の表示密度を上げる改修を進める過程で、以下の 3 点について生成 AI を利用した。

- 例題 2 のコードでシリアルプロッタの波形が粗くなる原因の診断
- 「等間隔サンプリング」と「整数 3 系列出力」へ方針転換する際の設計議論
- `micros()` の 32bit ラップアラウンドを扱うサンプリングループの書き方の確認

2. AI への指示（プロンプト）

AI へ送った指示のうち、主要なものを以下に示す。

例題2 (work3.ino) をシリアルプロッタで見ると波形がスカスカに見える。
原因を診断して、修正方針を箇条書きで挙げてほしい。
着目してほしい観点：
- `loop()` 内の処理時間 (`Serial.print` の回数・文字数)
- サンプリング周期が可変になっていないか
- 表示用データの型 (`float` vs `int`)
- プロッタのY軸の自動レンジが暴れる要因

整数3系列（生値・上限ガイド・下限ガイド）を等間隔で出す
HCK02_01.ino の雛形を書いてほしい。
条件：
- サンプリングは `delay()` ではなく `micros()` の差分で等間隔にする
- `micros()` は 32bit なのでラップアラウンドで詰まらないようにする
- 電圧変換や DC オフセット処理は今回は入れない
- コメントはあとで自分の言葉で説明できるよう簡潔に

3. AI の出力の使い方

- ☐ そのまま使った
- ☒ 一部修正して使った
- ☐ 参考にしただけ

→ 上で選んだ理由：

AI が挙げた診断・修正方針のうち、以下は妥当と判断して HCK02_01.ino に採用した：

- 例題 2 は `float` の電圧変換と日本語ラベル付き 4 系列出力で 1 サンプルあたりの送信バイト数が大きく、サンプリング周期が不安定になる → 出力を整数 3 系列 (d, 上限 1023, 下限 0) に絞り、ラベルも

省略

- `delay()` ではサンプリング周期が処理時間ぶん遅れて揺れる → `micros()` の目標時刻との差分で busy-wait し、周期 `SAMPLE_US = 150` を厳密に刻む
- `micros()` は約 70 分で 32bit ラップアラウンドする → `(int32_t)(micros() - nextSampleUs) < 0` の符号付き比較で安全に判定
- プロッタの Y 軸が自動レンジで暴れる → 毎サンプル $(1 \ll \text{RESOLUTION}) - 1$ と 0 をガイド線として出力し、Y 軸を 0-1023 に固定

一方で、AI から出てきた「電圧変換して中心を 0 にする案」「DC 追従フィルタを入れる案」は、今回の課題では「生の A/D 値をそのままプロットして波形の密度を見る」ことが目的なので、スコープ外として採用しなかった。

変数名・コメント・`SAMPLE_US` の値 ($150\mu\text{s} \approx \text{約 } 6.6\text{ kHz}$) は、AI 提案の雛形を下敷きに自分で書き直し／決め直している。

4. 正しいと確認した方法

[TODO: 実機テスト後に、具体的に観察した内容を記入する]

想定している確認手順：

- Arduino IDE でコンパイルが通ることを確認する
- シリアルモニタを 921600 bps で開き、`d,1023,0` 形式の 3 値が流れることを確認する
- シリアルプロッタに切り替え、上下にガイド線 (1023 と 0) が張られ、中央のトレースが A/D 値として動くことを確認する
- マイクに声をかけて中央のトレースが音の強弱に応じて振動し、例題 2 より波形が細かく見えることを観察する
- コードを 1 行ずつ読み、`nextSampleUs += SAMPLE_US` による等間隔サンプリングと、`(int32_t)` キャストでラップアラウンドを扱っている意味を自分の言葉で説明できる状態にする

5. 問題や間違い

- ☐ あった
- ☐ なかった

[TODO: 実機テスト後に、実際に気づいた問題や AI の誤り・自分が気づいた点を記入する]

検証予定ポイント：

- `SAMPLE_US = 150` は 921600 bps で「整数 3 系列＋カンマ＋改行」を送り切れるか（送信が間に合わない場合は周期を広げるか、ガイド出力を間引く）
- `Serial.println` 完了待ちがサンプリング周期を食いつぶしていないか（オシロ代わりにピン出力を入れて確認する案）
- `analogReadResolution(10)` が UNO R4 側で期待通りに効いているか（既定も 10 bit のため、冗長だが明示目的で残している）

6. 自分で工夫した点

- 例題 2 の「電圧換算 + DC 中心化 + 4 系列ラベル付き出力」から、今回の目的（シリアルプロッタでの波形表示密度を上げる）に必要な要素だけに絞り、生の A/D 値ベースの整数 3 系列に単純化した
- `delay()` ベースではなく、`nextSampleUs` を一定量進めて busy-wait する方式にして、`analogRead` やシリアル送信のばらつきを周期に持ち込まないようにした
- `micros()` のラップアラウンドを `int32_t` への符号付きキャストで扱い、ハマりやすい落とし穴を明示的にコメント化した
- プロッタの Y 軸暴れを抑えるため、毎サンプル上下ガイド ($(1 \ll \text{RESOLUTION}) - 1$ と 0) を常時出力する設計にした。上限値をリテラルではなく `RESOLUTION` から計算することで、量子化精度を変えた場合もコード 1 箇所の変更で追従できるようにしている

参考文献

- 生成 AI 利用申告書テンプレート（ハッカソン 1 講義資料, [references/lectures/](#))
- Arduino Reference: `analogRead`, `analogReadResolution`, `micros`, `Serial` <https://docs.arduino.cc/language-reference/>
- 例題 2 オリジナルコード: [work/shiozawa/work-0422/work3/work3.ino](#)